

AI PRODUCT PRESENTATION · M5 · PM for AI Projects

Log Analysis AI

ผู้ช่วยวิเคราะห์ Log และเฝ้าระวังระบบด้วย AI

ตรวจจับความผิดปกติ → วิเคราะห์ต้นเหตุ → แจ้งเตือนอัจฉริยะ → ให้คนยืนยัน

🔗 Live demo natpakanchamp.github.io/LogAnalysisAI

ผู้จัดทำ **ณัฐปคัลภ์ กันทะสร** (Natpakan Kanthasorn)

รหัส **CP250041** · CP Scholarship AI PM Track · True Digital Academy

ทำไมต้องมี Log Analysis AI

ระบบ Microservices ล่ม — แต่ไม่รู้เพราะอะไร

- log ไหลเข้ามา มหาศาล ข้าม service หลายตัว
- ความผิดปกติเป็นแบบ **multi-dimension** (หลายปัจจัยพร้อมกัน)
- **rule-based** เดิมจับไม่ได้ เพราะตั้งกฎครอบคลุมไม่ไหว
- วิศวกรต้อง ไล่ log เองหลายชั่วโมง กว่าจะหาต้นเหตุ

ผลกระทบ

DevOps / Backend	เสียเวลา ไล่ log ทีละ service
ผู้ใช้ปลายทาง	เจอ downtime / latency
ธุรกิจ	เสียรายได้ + ความเชื่อมั่น

AI ช่วยย่อเวลาหาต้นเหตุจากชั่วโมง เหลือไม่กี่นาที

ระบบรับ log แบบ real-time แล้วใช้ AI 3 ขั้นตอนทำงานต่อกัน:

1

Anomaly Detection

ตรวจจับจุดผิดปกติจาก log อัตโนมัติ (Predictive)

2

Classification

จัดหมวดต้นเหตุ + ชี้ service/commit ที่น่าสงสัย (Supervised)

3

Generative AI

สรุปสาเหตุเป็นภาษาคนให้วิศวกร (LLM)

+ มี **Redaction layer** ลบความลับก่อนเข้าโมเดล และ **Human-in-the-Loop** ให้คนยืนยันความเสี่ยง

โมเดล AI ที่ใช้ (ดึงจาก GitHub + Google)

โมเดล	ประเภท	หน้าที่	ที่มา
DeepLog (LSTM) ★	Anomaly Detection	ตรวจจับความผิดปกติ (โมเดลหลัก)	GitHub · PyTorch · เทรนเอง
Logistic Regression	Classification	จัดหมวดต้นเหตุ	scikit-learn · เทรนเอง
Gemini 2.5 Flash	Generative (LLM)	สรุปสาเหตุภาษาคน	Google API (ฟรี)
Drain3	Log Parsing	แยก log → template	GitHub logpai/Drain3

มี **rule-based** (regex) เป็นตัวเทียบ — ไม่ใช่ AI — เพื่อพิสูจน์ว่า AI ดีกว่าของเดิม

โมเดลดึงจาก Kaggle หรือเทรนเอง?

✓ ข้อสรุป: โมเดลหลัก (DeepLog) **เทรนเองในเครื่อง 100%** — ไม่ได้ดึงโมเดลสำเร็จรูปจาก Kaggle

โค้ด โมเดล

port โครงสร้าง LSTM จาก GitHub
donglee/logdeep (MIT) — เป็นสถาปัตยกรรม
ไม่ใช่ค่าน้ำหนักสำเร็จ

น้ำหนัก (Weights)

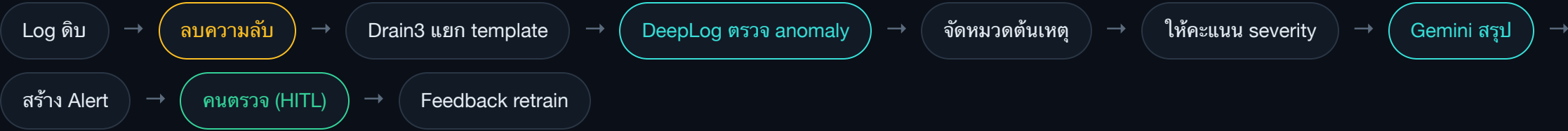
เทรนเองด้วย `scripts/train.py` → ได้ไฟล์
`models/model_bundle.pt`

บทบาทของ Kaggle

เป็นแค่แหล่ง **ชุดข้อมูล** HDFS (mirror ของ loghub)
— ไม่ใช่โมเดล

หลักฐานในโค้ด: `train.py` ทำ Drain แยก template + `DeepLog.fit()` เฉพาะ log ปกติ + เทรน classifier แล้ว `save_bundle()` · `download_data.py --hdfs` แค่มิर्फำ
แนะนำให้ไปโหลด dataset — ไม่มีการดาวน์โหลดหรือโหลดน้ำหนักโมเดลสำเร็จจากที่ใด · พิสูจน์ได้: นำโค้ดเดิมไป **เทรนข้ามข้อมูลจริง HDFS_v1** (3,600 sessions ทดสอบ จาก loghub) แล้วดู `num_candidates` → ที่ g=4: F1 **0.844** (จาก F1 0.252 ก่อนจูน) · precision **0.848** · high-severity recall **1.000** — ผ่านเกณฑ์ PRD

ข้อมูลไหลผ่านระบบยังไง



เข้า (INPUT)

log จากทุก microservice (error, latency, error-rate) แบบ real-time

ออก (OUTPUT)

Alert ที่บอก ความรุนแรง + ความมั่นใจ + สาเหตุที่เป็นไปได้ + service/commit ที่น่าสงสัย

AI ไม่ตัดสินคนเดียว — เคลสเสี่ยงให้คนยืนยัน

ใช้ **Pattern 2**: คนตรวจเฉพาะ alert ที่ถูก flag เท่านั้น (ไม่ตรวจทุกอัน) เจื่อนไขชัดเจน วัดได้:

Confidence < 0.65

โมเดลไม่มั่นใจ → ส่งคนตรวจ

AI ↔ Rule ขัดแย้ง

AI ว่าผิดปกติ แต่ rule เจียบ → ตรวจ

Heartbeat Deadlock

ระบบค้างแต่ไม่มี error log → ตรวจ

วิศวกรกด **ยอมรับ / ปฏิเสธ** → เก็บเป็น feedback ไปปรับปรุงโมเดล (ปิด loop)

วัดผลจริงบน 2 ชุดข้อมูล — ผ่านเกณฑ์ PRD ทั้งหมด

1.000

Recall (high-severity)

เป้า ≥ 0.90 · PASS

0.831

Precision (overall)

เป้า ≥ 0.70 · PASS

0.791

F1 (overall)

เป้า ≥ 0.77 · PASS

37

เคสที่ AI จับได้ แต่ rule พลาด

เป้า > 0 · PASS

การวัดด้านบน = ชุด Synthetic (ตัวชี้วัดหลัก PRD) · ตารางด้านล่าง = เทียบกับข้อมูลจริง HDFS

ชุดข้อมูล (test set)	Precision	Recall	F1	Recall (high-sev)	PRD
Synthetic · sample · g=2 · 540 sessions	0.831	0.755	0.791	1.000	PASS
Real HDFS_v1 · loghub · g=4 · 3,600 sessions	0.848	0.841	0.844	1.000	PASS

ทั้ง 2 ชุดผ่านเกณฑ์ PRD ครบ (จน `num_candidates` ต่อชุดข้อมูล) · rule-based เดิมได้ Recall เพียง 0.418 (synthetic) / 0.681 (HDFS) — AI จับได้มากกว่าชัดเจน · ตัวเลขทุกตัว
รันซ้ำ + ตรวจสอบด้วยมือได้

ติดตั้งและรันระบบ

1 ติดตั้ง (ครั้งเดียว)

```
python3 -m venv .venv && source .venv/bin/activate  
pip install -e ".[dev]"
```

2 สร้างข้อมูล + เทรนโมเดล

```
python scripts/download_data.py --sample  
python scripts/train.py --dataset sample
```

3 เปิดเว็บ Dashboard

```
uvicorn api.main:app --port 8000 # เปิด http://localhost:8000
```

+ (ทางเลือก) เปิด AI สรุปลภาษาคน — ใส่ Gemini API key ฟรีในไฟล์ .env:

```
GEMINI_API_KEY=AIza... # ขอฟรีที่ aistudio.google.com/apikey
```

ใช้งานหน้า Dashboard — ตั้งแต่เปิดจนกดยอมรับ

1 กรอง

กดปุ่ม "Needs review" ดูเฉพาะเคสที่ต้องตรวจ

2 อ่านการ์ด

ดูความรุนแรง · confidence · service/commit ที่น่าสงสัย

3 ดูหลักฐาน

กางดู log จริง (ลบความลับแล้ว)

4 ♦ Explain with AI

ให้ Gemini สรุปสาเหตุเป็นภาษาคน

5 ตัดสินใจ

กด ✓ ยอมรับ (เป็นปัญหาจริง) หรือ ✕ ปฏิเสธ (เตือนผิด)

6 ปิด loop

feedback ถูกบันทึกไปปรับปรุง โมเดล

🔗 ลองเล่นได้ทันที (ไม่ต้องติดตั้ง): natpakanchamp.github.io/LogAnalysisAI · วัดผลเอง: `python scripts/evaluate.py --dataset sample`

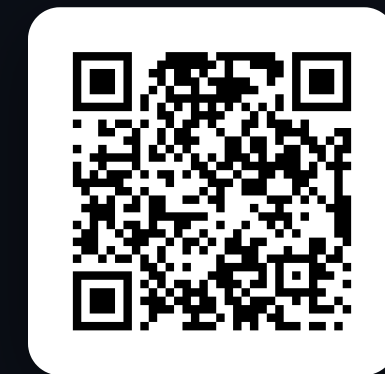
สรุป · SUMMARY

Log Analysis AI

เปลี่ยนการไล่ log หลายชั่วโมง ให้เหลือ "อ่าน + กดยืนยัน" ไม่กี่นาที

- ✓ AI 3 ชั้น: ตรวจจับ · จัดหมวด · สรุป
- ✓ ผ่านเกณฑ์ PRD ครบ — Recall เคสรุนแรง 1.000
- ✓ Human-in-the-Loop กันความผิดพลาด
- ✓ พิสูจน์ผลได้ รันซ้ำได้

ณัฐปคัลภ์ กันทะศรี · CP250041 · ผู้ช่วยวิเคราะห์ Log และเฝ้าระวังระบบด้วย AI



สแกนเพื่อลองเดโม

natpakanchamp.github.io/LogAnalysisAI